
BÁO CÁO BÀI TẬP VỀ NHÀ 6

KHẢO SÁT ĐIỀU KIỆN HỘI TỤ CỦA CÁC PHƯƠNG PHÁP LẬP JACOBI VÀ GAUSS-SEIDEL QUA MA TRẬN LẬP

Họ và tên: Nguyễn Trần Gia Bảo

MSSV: 23137003

Môn học: Vật lý tính toán (PHY10532)

Giảng viên: TS. Vũ Quang Tuyên

June 30, 2026

Contents

1	Giới thiệu và nội dung bài tập	2
2	Cơ sở lý thuyết	2
2.1	Hệ phương trình tuyến tính và tách ma trận	2
2.2	Phương pháp Jacobi và ma trận lặp T_J	3
2.3	Phương pháp Gauss–Seidel và ma trận lặp T_{GS}	3
2.4	Điều kiện hội tụ: bán kính phổ $\rho(T)$	3
2.5	Chéo trội nghiêm ngặt: điều kiện <i>đủ</i> , không phải <i>cần</i>	3
2.6	Tiêu chuẩn dừng	4
3	Phân tích chương trình	4
3.1	Các hàm tiện ích	4
3.1.1	Tính bán kính phổ	4
3.1.2	Kiểm tra chéo trội	4
3.1.3	Tách ma trận D , L , U và tính T_{GS} , T_J	5
3.2	Hàm Gauss–Seidel	5
3.3	Hàm Jacobi	6
4	Kết quả và thảo luận	7
4.1	Thực hành 1a: Hệ không chéo trội nhưng vẫn hội tụ	7
4.1.1	Câu a. Kiểm tra chéo trội nghiêm ngặt	7
4.1.2	Câu b. Ma trận T_{GS} và bán kính phổ	7
4.1.3	Câu c. Giải bằng Gauss–Seidel, $\varepsilon = 10^{-2}$	7
4.2	Thực hành 1b: Phản ví dụ Jacobi–tốt-hơn–Gauss–Seidel	8
4.2.1	Câu a. Ma trận T_J và bán kính phổ	8
4.2.2	Câu b. Giải bằng Jacobi, $\mathbf{x}_0 = \mathbf{0}$, $\varepsilon = 10^{-5}$	9
4.2.3	Câu c. Chứng minh $\rho(T_{GS}) = 2$ và kiểm chứng phân kỳ	9
4.2.4	Nhận xét tổng quát về hai hành vi trái ngược	10
5	Kết luận	10

1. Giới thiệu và nội dung bài tập

Ở báo cáo BTVN 5 ta đã thấy rằng với hệ phương trình có ma trận hệ số *chéo trội nghiêm ngặt*, cả Jacobi lẫn Gauss–Seidel đều hội tụ, và Gauss–Seidel thường nhanh hơn. Trực giác thông thường rút ra là: “Gauss–Seidel tốt hơn Jacobi vì dùng thông tin mới nhất”. Tuy nhiên, đây chỉ là *định lý đặc biệt*, không phải *định lý tổng quát*. Bài thực hành này đưa ra hai hệ phương trình nhằm củng cố công cụ lý thuyết chặt chẽ hơn – **bán kính phổ ρ của ma trận lặp** – và đồng thời chỉ ra một phản ví dụ rất “ngược” với trực giác thông thường. Cụ thể:

1. **Thực hành 1a.** Cho hệ

$$\begin{cases} x_1 - x_3 = 0.2 \\ -0.5x_1 + x_2 - 0.25x_3 = -1.425 \\ x_1 - 0.5x_2 + x_3 = 2 \end{cases}$$

Yêu cầu:

- Ma trận hệ số có phải là “strictly diagonally dominant” không?
- Xác định ma trận lặp T_{GS} và tính bán kính phổ $\rho(T_{GS})$.
- Dùng phương pháp Gauss–Seidel giải hệ với độ chính xác $\varepsilon = 10^{-2}$.

2. **Thực hành 1b.** Cho hệ

$$\begin{cases} x_1 + 2x_2 - 2x_3 = 7 \\ x_1 + x_2 + x_3 = 2 \\ 2x_1 + 2x_2 + x_3 = 5 \end{cases}$$

Yêu cầu:

- Xác định ma trận T_J và tính $\rho(T_J)$.
- Dùng Jacobi với $\mathbf{x}_0 = \mathbf{0}$ để giải hệ với $\varepsilon = 10^{-5}$.
- Chứng minh $\rho(T_{GS}) = 2$. Kiểm chứng Gauss–Seidel “thất bại” cho $\varepsilon = 10^{-5}$ sau 25 vòng lặp.

2. Cơ sở lý thuyết

2.1. Hệ phương trình tuyến tính và tách ma trận

Xét hệ

$$A\mathbf{x} = \mathbf{b}, \quad A \in \mathbb{R}^{n \times n}, \mathbf{x}, \mathbf{b} \in \mathbb{R}^n. \quad (1)$$

Để áp dụng các phương pháp lặp ta viết

$$A = D + L + U,$$

với D là phần đường chéo, L là phần tam giác dưới nghiêm ngặt, U là phần tam giác trên nghiêm ngặt. Điều kiện chung là $a_{ii} \neq 0$ với mọi i .

2.2. Phương pháp Jacobi và ma trận lặp T_J

Công thức lặp Jacobi:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right). \quad (2)$$

Viết dưới dạng ma trận:

$$\mathbf{x}^{(k+1)} = -D^{-1}(L + U) \mathbf{x}^{(k)} + D^{-1}\mathbf{b}.$$

Ma trận lặp:

$$\boxed{T_J = -D^{-1}(L + U)} \quad (3)$$

2.3. Phương pháp Gauss–Seidel và ma trận lặp T_{GS}

Công thức lặp Gauss–Seidel dùng ngay các giá trị mới vừa được cập nhật:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right). \quad (4)$$

Dạng ma trận:

$$(D + L)\mathbf{x}^{(k+1)} = -U\mathbf{x}^{(k)} + \mathbf{b} \implies \mathbf{x}^{(k+1)} = -(D + L)^{-1}U \mathbf{x}^{(k)} + (D + L)^{-1}\mathbf{b}.$$

Ma trận lặp:

$$\boxed{T_{GS} = -(D + L)^{-1}U} \quad (5)$$

2.4. Điều kiện hội tụ: bán kính phổ $\rho(T)$

Định lý (hội tụ của phép lặp tuyến tính): Phương pháp lặp tổng quát $\mathbf{x}^{(k+1)} = T\mathbf{x}^{(k)} + \mathbf{c}$ hội tụ với mọi $\mathbf{x}^{(0)}$ khi và chỉ khi

$$\rho(T) = \max_{\lambda \in \sigma(T)} |\lambda| < 1, \quad (6)$$

trong đó $\sigma(T)$ là tập các trị riêng (phổ) của T .

Ý nghĩa: sai số sau k bước $\mathbf{e}^{(k)} = T^k \mathbf{e}^{(0)}$. Với k đủ lớn, $\|T^k\| \sim \rho(T)^k$, nên dãy hội tụ khi và chỉ khi $\rho(T) < 1$. Ngoài ra, $\rho(T)$ càng nhỏ thì hội tụ càng nhanh: tốc độ hội tụ tiệm cận là $-\log_{10} \rho(T)$ chữ số chính xác trên mỗi vòng lặp.

2.5. Chéo trội nghiêm ngặt: điều kiện đủ, không phải cần

Điều kiện chéo trội nghiêm ngặt theo hàng:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \quad \forall i. \quad (7)$$

Định lý: Nếu Phương trình (7) thỏa, thì $\rho(T_J) < 1$ và $\rho(T_{GS}) < 1$; cả hai phương pháp hội tụ.

2.6. Tiêu chuẩn dừng

$$\text{err}^{(k)} = \max_{1 \leq i \leq n} |x_i^{(k+1)} - x_i^{(k)}| < \varepsilon. \quad (8)$$

Kèm theo đó là giới hạn số bước lặp tối đa $N_{\max}$ để tránh vòng lặp vô hạn khi phương pháp không hội tụ.

3. Phân tích chương trình

3.1. Các hàm tiện ích

3.1.1. Tính bán kính phổ

```

1 def spectral_radius(A):
2     eigenvalues = np.linalg.eigvals(A)
3     bankinhpho = max(abs(eigenvalues))
4     if bankinhpho < 1:
5         ondingh = True
6         print("He on dinh")
7     else:
8         ondingh = False
9         print("He khong on dinh")
10    return bankinhpho, ondingh

```

Giải thích:

- `np.linalg.eigvals(A)`: trả về vector trị riêng phức của A .
- $\rho(A) = \max_i |\lambda_i|$: dùng `abs()` cho cả trị riêng phức.
- Cờ `ondingh` đánh dấu $\rho < 1$ ($\Leftrightarrow$ phép lặp hội tụ).

3.1.2. Kiểm tra chéo trội

```

1 def diag_dominant(A):
2     n = A.shape[0]
3     sum_row = 0
4     for i in range(n):
5         for j in range(n):
6             if i != j:
7                 sum_row = sum_row + abs(A[i, j])
8             if abs(A[i, i]) > sum_row:
9                 print("Ma tran A co troi duong cheo nghiem ngat")
10                diag_dominant = 'strict'
11            elif abs(A[i, i]) >= sum_row:
12                print("Ma tran A co troi duong cheo")
13                diag_dominant = 'yes'
14            else:
15                diag_dominant = 'no'
16    return diag_dominant

```

3.1.3. Tách ma trận D , L , U và tính T_{GS} , T_J

```

1 def matran_diag(A):          return np.diag(np.diag(A))
2 def matran_duoi(A):          return np.tril(A, k=-1)
3 def matran_tren(A):          return np.triu(A, k=1)
4
5 def matran_gauss_seidel(A):
6     D = matran_diag(A); L = matran_duoi(A); U = matran_tren(A)
7     T_gs = np.dot(np.linalg.inv(D + L), -U)
8     return T_gs
9
10 def matran_jacobi(A):
11     D = matran_diag(A); L = matran_duoi(A); U = matran_tren(A)
12     T_jacobi = np.dot(np.linalg.inv(D), L + U)
13     return T_jacobi

```

Giải thích:

- `np.diag(np.diag(A))`: trích chéo và ráp thành ma trận đường chéo.
- `np.tril(A, k=-1)`: phần tam giác dưới *không* chứa đường chéo.
- `np.triu(A, k=1)`: phần tam giác trên *không* chứa đường chéo.
- Công thức $T_{GS} = -(D + L)^{-1}U$ và $T_J = -D^{-1}(L + U)$.
- **Ghi chú:** Hàm `matran_jacobi` trong code thiếu dấu trừ so với công thức Phương trình (3). Tuy nhiên *dấu không ảnh hưởng đến bán kính phổ*, vì các trị riêng chỉ đổi dấu còn $|\lambda|$ không đổi.

3.2. Hàm Gauss–Seidel

```

1 def Gauss_Seidel(N_max, epsilon, A, B):
2     n = len(B)
3     x_old = n * [0]
4     x_new = n * [0]
5     err_mang = n * [100]
6     err = np.max(err_mang)
7     x_matrix = []
8     err_matrix = []
9     x_matrix.append(x_old[:])
10    for q in range(N_max):
11        if err > epsilon:
12            for i in range(n):
13                tong1 = tong2 = 0
14                for j in range(i):
15                    tong1 = tong1 + A[i][j] * x_new[j]
16                for j in range(i+1, n):
17                    tong2 = tong2 + A[i][j] * x_old[j]
18                x_new[i] = 1 / A[i][i] * (-tong1 - tong2 + B[i])
19                err_mang[i] = np.abs(x_new[i] - x_old[i])
20            err = np.max(err_mang[:])

```

```

21         x_matrix.append(x_new[:])
22         err_matrix.append(err)
23         x_old[:] = x_new[:]
24     else:
25         break
26     return q, x_matrix, err_matrix

```

Giải thích:

- tong1 chứa $\sum_{j<i} a_{ij} x_j^{(k+1)}$ dùng giá trị mới.
- tong2 chứa $\sum_{j>i} a_{ij} x_j^{(k)}$ dùng giá trị cũ.
- Đặc điểm đặc trưng của Gauss–Seidel: tong1 đọc x_new[j] (đã được cập nhật trong vòng lặp hiện tại), còn tong2 đọc x_old[j] (chưa cập nhật).
- Tiêu chuẩn dừng là chuẩn vô cùng: $\max |x_i^{(k+1)} - x_i^{(k)}| < \varepsilon$.

3.3. Hàm Jacobi

```

1 def Jacobi(N_max, epsilon, A, B):
2     n = len(B)
3     x_old = n * [0]
4     x_new = n * [0]
5     err_mang = n * [100]
6     err = np.max(err_mang)
7     x_matrix = []
8     err_matrix = []
9     x_matrix.append(x_old[:])
10    for q in range(N_max):
11        if err > epsilon:
12            for i in range(n):
13                tong = 0
14                for j in range(n):
15                    if i != j:
16                        tong = tong + A[i][j] * x_old[j]
17                x_new[i] = 1 / A[i][i] * (-tong + B[i])
18                err_mang[i] = np.abs(x_new[i] - x_old[i])
19            err = np.max(err_mang)
20            x_matrix.append(x_new[:])
21            err_matrix.append(err)
22            x_old[:] = x_new[:]
23        else:
24            break
25    return q, x_matrix, err_matrix

```

Khác biệt với Gauss–Seidel: toàn bộ tổng chỉ dùng x_old[j] với $j \neq i$. Không có tách tong1/tong2 vì không phân biệt “đã cập nhật” hay “chưa cập nhật”. Jacobi đợi hết vòng ngoài mới đồng bộ cập nhật $x_old \leftarrow x_new$.

4. Kết quả và thảo luận

4.1. Thực hành 1a: Hệ không chéo trội nhưng vẫn hội tụ

4.1.1. Câu a. Kiểm tra chéo trội nghiêm ngặt

Hệ số:

$$A = \begin{pmatrix} 1 & 0 & -1 \\ -0.5 & 1 & -0.25 \\ 1 & -0.5 & 1 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 0.2 \\ -1.425 \\ 2 \end{pmatrix}.$$

Kiểm tra từng hàng theo Phương trình (7):

- Hàng 1: $|a_{11}| = 1$ vs $|0| + |-1| = 1 \implies \text{bằng nhau}$ (chéo trội yếu, không strict).
- Hàng 2: $|a_{22}| = 1$ vs $|-0.5| + |-0.25| = 0.75 \implies \text{thoả strict}$.
- Hàng 3: $|a_{33}| = 1$ vs $|1| + |-0.5| = 1.5 \implies \text{không thoả}$ ($1 < 1.5$).

Kết luận: Ma trận A không strictly diagonally dominant. Theo định lý chéo trội, ta không thể kết luận ngay Jacobi/Gauss–Seidel hội tụ; phải kiểm tra bằng bán kính phổ.

4.1.2. Câu b. Ma trận T_{GS} và bán kính phổ

Tính trực tiếp bằng công thức Phương trình (5):

$$D + L = \begin{pmatrix} 1 & 0 & 0 \\ -0.5 & 1 & 0 \\ 1 & -0.5 & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 0 & 0 & -1 \\ 0 & 0 & -0.25 \\ 0 & 0 & 0 \end{pmatrix}.$$

Sau khi tính:

$$T_{GS} = -(D + L)^{-1}U = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 0 & 0.75 \\ 0 & 0 & -0.625 \end{pmatrix}.$$

Ma trận T_{GS} là ma trận tam giác trên thưa, nên trị riêng chính là các phần tử đường chéo: $\lambda_1 = \lambda_2 = 0, \lambda_3 = -0.625$. Vậy

$$\boxed{\rho(T_{GS}) = 0.625 < 1} \quad (9)$$

$\implies$ Gauss–Seidel hội tụ cho dù ma trận không strict dominant. Điều này khẳng định rằng chéo trội chỉ là điều kiện đủ, không cần.

4.1.3. Câu c. Giải bằng Gauss–Seidel, $\varepsilon = 10^{-2}$

Nghiệm đúng (kiểm chứng bằng `np.linalg.solve`): $\mathbf{x}^* = (0.9, -0.8, 0.7)^T$.

Kết quả lặp, hội tụ sau **13 vòng lặp**:

k	$x_1^{(k)}$	$x_2^{(k)}$	$x_3^{(k)}$	$\text{err}^{(k)}$
1	0.200000000	-1.325000000	1.137500000	1.33×10^0
2	1.337500000	-0.471875000	0.426562500	1.14×10^0
3	0.626562500	-1.005078125	0.870898438	7.11×10^{-1}
4	1.070898438	-0.671826172	0.593188477	4.44×10^{-1}
5	0.793188477	-0.880108643	0.766757202	2.78×10^{-1}
6	0.966757202	-0.749932098	0.658276749	1.74×10^{-1}
7	0.858276749	-0.831292439	0.726077032	1.08×10^{-1}
8	0.926077032	-0.780442226	0.683701855	6.78×10^{-2}
9	0.883701855	-0.812223609	0.710186341	4.24×10^{-2}
10	0.910186341	-0.792360245	0.693633537	2.65×10^{-2}
11	0.893633537	-0.804774847	0.703979039	1.66×10^{-2}
12	0.903979039	-0.797015721	0.697513100	1.03×10^{-2}
13	0.897513100	-0.801865175	0.701554312	6.47×10^{-3}

Table 1: Lịch sử lặp Gauss–Seidel cho Thực hành 1a. Dòng đậm là bước dừng khi $\text{err} < \varepsilon = 10^{-2}$.

Biện luận:

- Sai số giảm *xấp xỉ đều theo tỉ lệ hình học* với công bội $\approx 0.625 = \rho(T_{GS})$. Chẳng hạn $\text{err}^{(10)}/\text{err}^{(9)} \approx 2.65/4.24 \approx 0.625$. Đây là biểu hiện trực tiếp của định lý: tốc độ hội tụ tiệm cận đúng bằng $\rho(T_{GS})$.
- Để giảm sai số đi 10 lần cần khoảng $\log_{10}(10)/\log_{10}(1/0.625) = 1/0.204 \approx 4.9$ vòng lặp. Quan sát thực tế: từ vòng 2 (1.14) xuống vòng 7 (0.108) đúng ≈ 10 lần sau 5 bước – khớp lý thuyết.
- Nghiệm cuối (0.8975, -0.8019, 0.7016) sai lệch so với nghiệm đúng cỡ 10^{-2} , phù hợp với ngưỡng $\varepsilon = 10^{-2}$.
- Kết luận quan trọng:** Tuy A không chéo trội, Gauss–Seidel vẫn hội tụ nhờ $\rho(T_{GS}) = 0.625 < 1$. Minh chứng rằng *bán kính phổ* là *tiêu chuẩn duy nhất chặt chẽ*.

4.2. Thực hành 1b: Phản ví dụ Jacobi–tốt-hơn–Gauss–Seidel

4.2.1. Câu a. Ma trận T_J và bán kính phổ

Hệ số:

$$A = \begin{pmatrix} 1 & 2 & -2 \\ 1 & 1 & 1 \\ 2 & 2 & 1 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 7 \\ 2 \\ 5 \end{pmatrix}.$$

Vì $D = I$ nên tính đơn giản: $T_J = -D^{-1}(L + U) = -(L + U)$:

$$T_J = \begin{pmatrix} 0 & -2 & 2 \\ -1 & 0 & -1 \\ -2 & -2 & 0 \end{pmatrix}.$$

(Code trong báo cáo trả về $D^{-1}(L + U)$ không có dấu trừ; dấu không ảnh hưởng bán kính phổ.)

Tính trị riêng: Sử dụng `np.linalg.eigvals`, output là ba số xấp xỉ 0 trong phạm vi sai số máy ($\sim 10^{-5}$ do ma trận có cấu trúc rất “nhảy”). Điều này gợi ý T_J là ma trận nilpotent.

Chúng minh $T_J^3 = 0$ bằng tay:

$$T_J^2 = \begin{pmatrix} 0 & -2 & 2 \\ -1 & 0 & -1 \\ -2 & -2 & 0 \end{pmatrix}^2 = \begin{pmatrix} -2 & -4 & 2 \\ 2 & 4 & -2 \\ 2 & 4 & -2 \end{pmatrix}.$$

Tiếp tục $T_J^3 = T_J^2 \cdot T_J$ cho ra ma trận không. Vậy T_J nilpotent bậc ≤ 3 . Kết quả:

$$\boxed{\rho(T_J) = 0} \quad (10)$$

Điều này rất đặc biệt: $\rho = 0$ có nghĩa Jacobi sẽ *hội tụ sau tối đa 3 vòng lặp* đến nghiệm chính xác toán học (bỏ qua sai số làm tròn máy).

4.2.2. Câu b. Giải bằng Jacobi, $\mathbf{x}_0 = \mathbf{0}$, $\varepsilon = 10^{-5}$

Nghiệm đúng: $\mathbf{x}^* = (1, 2, -1)^T$.

k	$x_1^{(k)}$	$x_2^{(k)}$	$x_3^{(k)}$	$\text{err}^{(k)}$
1	7.000000	2.000000	5.000000	7.00×10^0
2	13.000000	-10.000000	-13.000000	1.80×10^1
3	1.000000	2.000000	-1.000000	1.20×10^1
4	1.000000	2.000000	-1.000000	0.0

Table 2: Lịch sử lặp Jacobi cho Thực hành 1b. Nghiệm đạt giá trị chính xác ngay sau 3 bước (đến vòng 4 sai số = 0).

Biện luận:

- Sau 3 bước, nghiệm trùng khớp *tuyệt đối* với nghiệm đúng $(1, 2, -1)$. Điều này khớp hoàn hảo với $T_J^3 = 0$: sai số $\mathbf{e}^{(3)} = T_J^3 \mathbf{e}^{(0)} = \mathbf{0}$.
- Ở vòng 4, $\text{err} = 0 < 10^{-5}$ nên vòng lặp dừng. Jacobi hoàn toàn không cần chạy tới hàng nghìn bước như nhiều hệ thông thường.
- Đáng chú ý: ở các vòng 1–2, nghiệm tạm thời *xa* nghiệm đúng (giá trị ± 13), nhưng vẫn “rơi đúng” về nghiệm ở vòng 3. Đây là điểm rất khác các hệ chéo trội (vốn giảm đơn điệu); với ma trận nilpotent sự giảm sai số không nhất thiết đơn điệu nhưng cuối cùng dừng tuyệt đối sau ≤ 3 bước.

4.2.3. Câu c. Chứng minh $\rho(T_{GS}) = 2$ và kiểm chứng phân kỳ

Tính T_{GS} : Với A đã cho:

$$D + L = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 2 & 2 & 1 \end{pmatrix}, \quad (D + L)^{-1} = \begin{pmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ 0 & -2 & 1 \end{pmatrix}.$$

$$U = \begin{pmatrix} 0 & 2 & -2 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix}, \quad T_{GS} = -(D + L)^{-1}U = \begin{pmatrix} 0 & -2 & 2 \\ 0 & 2 & -3 \\ 0 & 0 & 2 \end{pmatrix}.$$

T_{GS} là ma trận tam giác trên nên trị riêng chính là đường chéo: $\lambda_1 = 0, \lambda_2 = \lambda_3 = 2$. Vậy

$$\boxed{\rho(T_{GS}) = 2} \quad (\text{khớp với đề bài}). \quad (11)$$

Vì $\rho(T_{GS}) = 2 > 1$, Gauss–Seidel *phân kỳ* với mọi giá trị khởi tạo $\neq \mathbf{x}^*$.

Kiểm chứng số: Chạy Gauss–Seidel với $\varepsilon = 10^{-5}$ và $N_{\max} = 25$:

k	$x_1^{(k)}$	$x_2^{(k)}$	$x_3^{(k)}$	$\text{err}^{(k)}$
1	7.00×10^0	-5.00×10^0	1.00×10^0	7.0×10^0
2	1.90×10^1	-1.80×10^1	3.00×10^0	1.3×10^1
3	4.90×10^1	-5.00×10^1	7.00×10^0	3.2×10^1
5	2.89×10^2	-3.02×10^2	3.10×10^1	1.76×10^2
10	1.69×10^4	-1.74×10^4	1.02×10^3	9.47×10^3
15	7.86×10^5	-8.03×10^5	3.28×10^4	4.26×10^5
20	3.30×10^7	-3.36×10^7	1.05×10^6	1.76×10^7
25	1.31×10^9	-1.33×10^9	3.36×10^7	6.88×10^8

Table 3: Lịch sử “lặp” Gauss–Seidel cho Thực hành 1b (trích một số bước). Nghiệm phân kỳ cực mạnh, sau 25 bước đã lên tới $\sim 10^9$, cách xa nghiệm đúng $(1, 2, -1)$.

Biện luận:

- Sau 25 vòng, $|x_1| \approx 1.3 \times 10^9$ trong khi nghiệm đúng $x_1^* = 1$. Sai số bùng nổ.
- Tỉ số tăng của $\text{err}^{(k+1)}/\text{err}^{(k)}$ tiệm cận $2 = \rho(T_{GS})$. Ví dụ: $\text{err}^{(25)}/\text{err}^{(24)} \approx 6.88 \times 10^8 / 3.31 \times 10^8 \approx 2.08$. Khớp dự đoán lý thuyết $\|\mathbf{e}^{(k+1)}\| \sim \rho \|\mathbf{e}^{(k)}\|$.
- So sánh với Jacobi: cùng hệ, Jacobi kết thúc sau 3 bước với nghiệm chính xác; Gauss–Seidel sau 25 bước đã sai lệch cỡ 10^9 . Đây là phản ví dụ dứt khoát cho quan niệm “GS luôn tốt hơn Jacobi”.

4.2.4. Nhận xét tổng quát về hai hành vi trái ngược

- Lý do bản chất:** Hai ma trận lặp T_J và T_{GS} là các phép biến đổi khác hẳn nhau; không có mối liên hệ đơn điệu giữa $\rho(T_J)$ và $\rho(T_{GS})$ ngoại trừ các trường hợp ma trận có tính chất đặc biệt. Với ma trận tổng quát, việc “dùng giá trị mới ngay lập tức” có thể *khuếch đại sai số* thay vì dập tắt nó.
- Hai ví dụ 1a và 1b cũng có thông điệp: trước khi áp dụng Jacobi hoặc Gauss–Seidel, *luôn luôn* nên tính $\rho(T)$ để biết liệu phép lặp có hội tụ và tốc độ ra sao.

5. Kết luận

Qua hai thực hành, các điểm chính rút ra như sau:

1. **Điều kiện hội tụ chặt chẽ duy nhất** của các phương pháp lặp tuyến tính dạng $\mathbf{x}^{(k+1)} = T\mathbf{x}^{(k)} + \mathbf{c}$ là $\rho(T) < 1$. Chéo trội nghiêm ngặt chỉ là điều kiện *đủ*, không phải *cần*.
2. **Thực hành 1a** minh họa một hệ *không* chéo trội nhưng vẫn hội tụ dưới Gauss–Seidel vì $\rho(T_{GS}) = 0.625 < 1$. Hội tụ sau 13 vòng với $\varepsilon = 10^{-2}$, tốc độ giảm sai số theo tỉ lệ hình học công bội $\approx \rho$ – khớp tuyệt đối với lý thuyết.
3. **Thực hành 1b** là một phản ví dụ kinh điển: $\rho(T_J) = 0$ (ma trận lặp nilpotent bậc 3) khiến Jacobi đạt nghiệm *chính xác tuyệt đối sau 3 vòng lặp*, trong khi $\rho(T_{GS}) = 2$ khiến Gauss–Seidel *phân kỳ* với sai số nhân đôi mỗi bước, sau 25 bước đã lên tới $\sim 10^9$. Trực giác “Gauss–Seidel luôn tốt hơn Jacobi” là *sai* trong trường hợp tổng quát.
4. **Quy tắc khi tính:** Trước khi chạy Jacobi hay Gauss–Seidel, luôn tính $\rho(T_J), \rho(T_{GS})$; chọn phương pháp có ρ nhỏ hơn; nếu cả hai ≥ 1 thì chuyển sang kỹ thuật khác (phương trình chuẩn tắc như BTVN 5, khử Gauss trực tiếp, SOR với ω tối ưu, v.v.).